

JAVA PLACEMENT MASTER SERIES • HANDBOOK 1

Java Foundations & Object-Oriented Programming

From JVM internals to the four pillars of OOP — placement-ready Java, start to finish

★ JDK / JRE / JVM Deep Dive

★ Java 8-21 Version Guide

★ Full OOP Mastery

★ Top 50 Interview Q&A

For MCA / B.Tech / B.E. Placement Preparation

Java Developer • Backend Roles • Spring Boot Prep

Table of Contents

- 1 Introduction to Java
- 2 Java Features
- 3 Java Architecture (JDK, JRE, JVM)
- 4 Java Versions
- 5 Java Fundamentals
- 6 Control Flow
- 7 Methods
- 8 Object-Oriented Programming
- 9 Java Keywords
- 10 Advanced OOP Concepts
- 11 Frequently Confused Concepts
- 12 Java in Real Projects
- 13 Placement Focus Summary
- 14 Common Java Mistakes
- 15 Final Revision

Tip: If you're short on time, revise **Part 15 (Final Revision)** and **Part 11 (Frequently Confused Concepts)** first — they compress the entire handbook into rapid-fire tables and a Top 50 Q&A list.

How to use this handbook

This is **Handbook 1** of the Java Placement Master Series, covering Java Foundations and Object-Oriented Programming end to end — from JVM internals through the four pillars of OOP. Read it top to bottom once, then use **Part 15 (Final Revision)** as your fast recap before interviews.

Importance tags used throughout:

- ★★★★★ Must Know for Placements
- ★★★★ Frequently Asked
- ★★ Medium Importance

Part 1 — Introduction to Java

History and Why Java Was Created

Java was created by **James Gosling** and team at **Sun Microsystems**, released in **1995**. It started as a project called "Oak," originally meant for smart consumer electronics (set-top boxes, appliances). The team needed a language that was:

- **Platform-independent** (the same code should run on any device)
- **Reliable** (no crashes from bad memory handling)
- **Simple** compared to C++

When the internet exploded in the mid-90s, Java pivoted to become the language for **portable, network-aware applications** — and this is exactly why "Write Once, Run Anywhere" became its slogan.

Evolution of Java

```
1995 → Java 1.0 (Oak renamed to Java)
1998 → Java 2 (J2SE, J2EE, J2ME editions introduced)
2004 → Java 5 (Generics, Annotations, Enums)
2014 → Java 8 (Lambdas, Streams) – biggest update ever
2017 → Java 9 (Module System)
2018+ → 6-month release cycle begins (Java 10, 11, 12...)
2021 → Java 17 (LTS, Sealed Classes, Records)
2023 → Java 21 (LTS, Virtual Threads, Pattern Matching)
```

Why Companies Still Use Java ★★★★★

Reason	Explanation
Maturity	25+ years of battle-tested libraries and frameworks
Backward compatibility	Old Java code still runs on new JVMs
Huge ecosystem	Spring, Hibernate, Maven, Gradle — massive tooling
Performance	JIT compilation makes Java fast for large-scale systems
Talent pool	Millions of trained developers available to hire
Enterprise trust	Banks, insurance, e-commerce run on Java at massive scale

Industry demand: Java remains one of the top 3 languages for backend/enterprise roles worldwide, especially with **Spring Boot** dominating microservices development.

Java Editions

Edition	Full Form	Used For
Java SE	Standard Edition	Core Java — the foundation every developer learns first
Jakarta EE	(formerly Java EE) Enterprise Edition	Large-scale enterprise apps — web servers, servlets, REST APIs
Java ME	Micro Edition	Embedded/small devices (largely legacy today, replaced by Android/IoT-specific stacks)

*This handbook focuses on **Java SE** — the foundation required before Jakarta EE/Spring Boot.*

Part 2 — Java Features

★★★★★

Every one of these is a **very common interview question**: "What are the features of Java?"

Feature	What It Means	Why It Matters
Platform Independent	Compiled code (bytecode) runs on any OS with a JVM	One codebase, deploy anywhere — cloud, servers, devices
Object-Oriented	Everything is modeled around classes and objects	Enables large, maintainable, reusable codebases
Robust	Strong memory management, exception handling, type checking	Fewer crashes; the compiler catches many bugs early
Secure	No explicit pointers, bytecode verification, sandboxed execution	Reduces risk of memory corruption and malicious code
Portable	No implementation-dependent features (e.g., <code>int</code> is always 32-bit)	Code behaves identically across platforms
Architecture Neutral	Bytecode format is not tied to any specific CPU architecture	Same <code>.class</code> file runs on x86, ARM, etc.
Dynamic	Can load classes at runtime	Enables plugins, reflection, dynamic linking
Distributed	Built-in support for networking (RMI, sockets, URL classes)	Easy to build networked/distributed applications
Multithreaded	Native support for concurrent execution via threads	Efficient use of multi-core CPUs, responsive applications
High Performance	JIT (Just-In-Time) compiler converts bytecode to native machine code at runtime	Near-native speed despite being "interpreted"
Automatic Garbage Collection	JVM automatically frees unused memory	Developers don't manually manage memory (unlike C/C++)

Platform Independence — how it actually works



The **same** `.class` file runs everywhere — only the JVM implementation differs per OS, not your code.

Interview Questions — Part 1-2

Q1. Why is Java called platform-independent when the JVM itself is platform-dependent? ★★★★★

- **Ideal Answer:** Java source code compiles to an intermediate format called bytecode, which is not tied to any OS or CPU. The JVM (which IS platform-specific — a different JVM binary exists for Windows, Linux, macOS) interprets/executes this same bytecode. So the code is portable even though the JVM that runs it is not.

- **Why asked:** Tests whether the candidate understands the actual mechanism, not just the marketing slogan "Write Once, Run Anywhere."
- **Common mistake:** Saying "Java runs the same on every machine" without mentioning bytecode or the JVM's role.
- **Difficulty:** Basic

Q2. Is Java 100% Object-Oriented? ★★★★★

- **Ideal Answer:** No. Java has 8 primitive types (`int` , `char` , `boolean` , etc.) that are not objects, which violates strict OOP purity. Truly pure OOP languages (like Smalltalk) treat even numbers as objects.
- **Why asked:** Tests broader awareness beyond textbook claims.
- **Difficulty:** Intermediate

Q3. How does Java achieve "Robustness"? ★★★★★

- **Ideal Answer:** Through strong compile-time type checking, automatic garbage collection (no dangling pointers), exception handling (forces error handling instead of silent crashes), and the absence of explicit pointer arithmetic (which is a common source of bugs in C/C++).
- **Difficulty:** Intermediate

Part 3 — Java Architecture



This is one of the **most-asked conceptual topics** in Java interviews: "Explain JDK, JRE, and JVM."

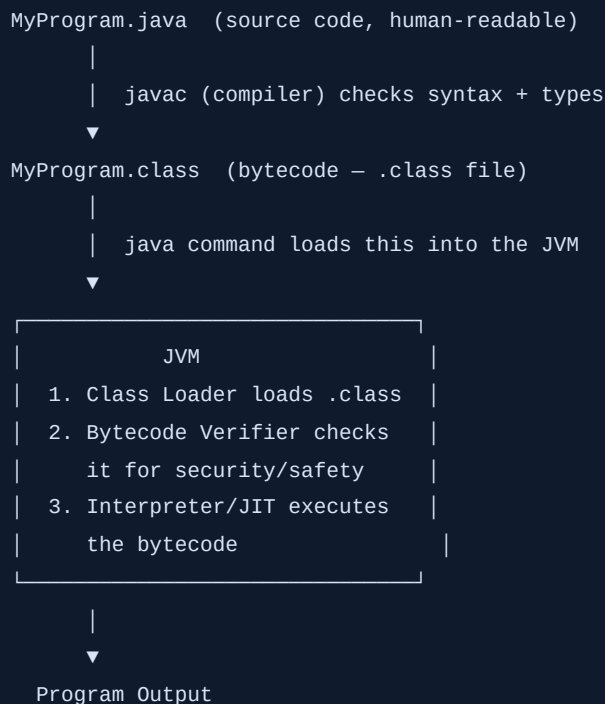
The Big Picture



Component	Full Form	Contains	Used By
JVM	Java Virtual Machine	Class loader, bytecode interpreter, JIT compiler, memory areas	Runs <code>.class</code> files
JRE	Java Runtime Environment	JVM + standard libraries (<code>java.lang</code> , <code>java.util</code> , etc.)	End users who only need to <i>run</i> Java apps
JDK	Java Development Kit	JRE + development tools (<code>javac</code> , debugger, <code>javadoc</code>)	Developers who <i>write and compile</i> Java code

Quick memory trick: $JDK \supset JRE \supset JVM$ (each one contains the previous, like Russian nesting dolls).

Compilation and Execution Process



Class Loader ★★★★★

The **Class Loader** is the JVM subsystem responsible for **dynamically loading** `.class` files into memory when needed (not all at once).

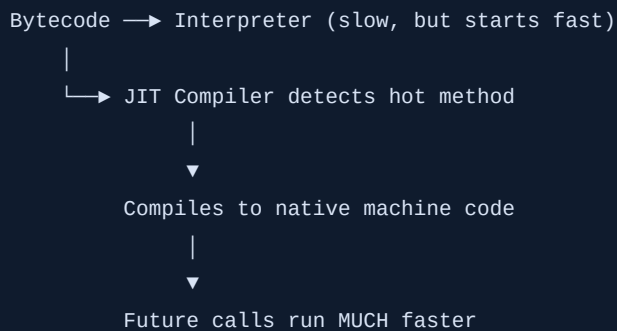
Three built-in loaders, forming a hierarchy (delegation model): 1. **Bootstrap Class Loader** — loads core Java classes (`java.lang.*`, etc.) from the JDK's core libraries 2. **Extension/Platform Class Loader** — loads classes from extension directories 3. **Application Class Loader** — loads classes from your application's classpath

Delegation principle: Before loading a class itself, each loader first asks its **parent** to try loading it — this prevents core classes like `java.lang.String` from being overridden accidentally.

JIT Compiler (Just-In-Time) ★★★★★

The JVM can run bytecode in two ways:

- **Interpretation** — reads and executes bytecode line by line (slower, but starts immediately)
- **JIT Compilation** — detects "hot" code (methods called repeatedly) and compiles it directly to **native machine code** at runtime, so future calls run at near-native speed



This is why Java is described as having "the best of both worlds" — fast startup (interpretation) plus fast steady-state performance (JIT).

Write Once, Run Anywhere (WORA) ★★★★★

Already illustrated in Part 2 — the key insight for interviews: **bytecode is portable, the JVM is not**. The JVM is the platform-specific "translator" that makes portability possible.

Interview Questions — Part 3

Q1. What is the difference between JDK, JRE, and JVM? ★★★★★ Frequently Asked

- **Ideal Answer:** JVM executes bytecode. JRE = JVM + standard libraries needed to run Java programs. JDK = JRE + development tools (compiler, debugger) needed to write and compile Java programs. JDK is a superset of JRE, which is a superset of JVM.
- **Why asked:** The single most common "day one" Java interview question — tests basic architectural understanding.
- **Common mistake:** Saying they're "the same thing" or confusing which one contains which.
- **Difficulty:** Basic

Q2. Can Java bytecode run without a JVM? ★★★

- **Ideal Answer:** No. Bytecode is not native machine code — it needs the JVM to either interpret it or JIT-compile it into instructions the actual CPU can execute.
- **Difficulty:** Basic

Q3. Explain the role of the Class Loader with the delegation model. ★★★★★

- **Ideal Answer:** The Class Loader loads `.class` files into JVM memory on demand. Java uses a hierarchical delegation model (Bootstrap → Extension → Application) where each loader asks its parent to load a class first, ensuring core classes can't be maliciously overridden and avoiding duplicate loading.
- **Why asked:** Tests deeper JVM internals knowledge, common in mid-to-senior interviews.
- **Difficulty:** Advanced

Part 4 — Java Versions



Java moved to a **6-month release cycle** starting Java 10, with **LTS (Long-Term Support)** versions released periodically (Java 8, 11, 17, 21...). Interviewers expect you to know what changed in the major LTS versions.

Version Comparison Table

Version	Year	Key Features	Placement Importance
Java 5	2004	Generics, Enums, Annotations, Autoboxing, Enhanced for-loop, Varargs	☆☆☆ Foundational — generics still core today
Java 7	2011	Try-with-resources, Diamond operator (<>), Switch on Strings	☆☆☆ Legacy code familiarity
Java 8	2014	Lambdas, Streams API, Functional Interfaces, Optional, new Date/Time API, Default methods in interfaces	☆☆☆☆☆ Most important version — asked constantly
Java 11	2018 (LTS)	var for local variables, new String methods, HTTP Client API, single-file source execution	☆☆☆☆☆ Common LTS baseline in industry
Java 17	2021 (LTS)	Sealed Classes, Records, Pattern Matching for instanceof, Text Blocks	☆☆☆☆☆ Widely adopted current LTS
Java 21	2023 (LTS)	Virtual Threads (Project Loom), Pattern Matching for switch, Record Patterns, Sequenced Collections	☆☆☆☆☆ Latest LTS — increasingly asked in fresh interviews

Java 8 — Why It's the Most Important Version ☆☆☆☆☆

```
// Before Java 8: verbose anonymous class
Runnable r1 = new Runnable() {
    public void run() { System.out.println("Running"); }
};

// Java 8: Lambda expression
Runnable r2 = () -> System.out.println("Running");

// Streams API - functional-style data processing
List<Integer> nums = List.of(1, 2, 3, 4, 5);
int sum = nums.stream()
    .filter(n -> n % 2 == 0)
    .mapToInt(Integer::intValue)
    .sum();    // sum = 6 (2 + 4)
```

Java 8 introduced **functional programming concepts** into Java — this fundamentally changed how modern Java code is written and is a near-guaranteed interview topic.

```
// Records (Java 17) - concise immutable data classes
record Point(int x, int y) {} // auto-generates constructor, getters, equals, hashCode, toString

// Pattern Matching for instanceof (Java 17)
if (obj instanceof String s) {
    System.out.println(s.length()); // 's' is auto-cast, no explicit (String) cast needed
}

// Pattern Matching for switch (Java 21)
String result = switch (obj) {
    case Integer i -> "Int: " + i;
    case String s -> "String: " + s;
    default -> "Unknown";
};

// Virtual Threads (Java 21) - lightweight threads for massive concurrency
Thread.startVirtualThread(() -> System.out.println("Running on a virtual thread"));
```

Interview Questions — Part 4

Q1. What changed in Java 8, and why was it a big deal? ★★★★★ Frequently Asked

- **Ideal Answer:** Java 8 introduced Lambda expressions and the Streams API, bringing functional programming into Java. It also added the `Optional` class (to reduce `NullPointerException`s), default/static methods in interfaces, and a new Date/Time API (`java.time`). It's considered the biggest shift in how Java code is written since the language's creation.
- **Why asked:** Nearly universal in Java interviews — signals whether the candidate has kept up with modern Java.
- **Difficulty:** Intermediate

Q2. What are Virtual Threads (Java 21) and why do they matter? ★★★★★

- **Ideal Answer:** Virtual threads are lightweight threads managed by the JVM (not the OS), allowing applications to run millions of concurrent tasks without the heavy memory/context-switching cost of traditional OS threads. They make it easy to write simple blocking-style code that scales like async code.
- **Why asked:** Tests awareness of cutting-edge Java, increasingly relevant for high-concurrency backend/microservices roles.
- **Difficulty:** Advanced

Q3. What is a Record, and how is it different from a regular class? ★★★★★

- **Ideal Answer:** A `record` is a special, concise class type (Java 17+) for immutable data carriers. It automatically generates a constructor, getters, `equals()` , `hashCode()` , and `toString()` based on its declared fields, eliminating boilerplate.
- **Difficulty:** Intermediate

Part 5 — Java Fundamentals



Variables and Data Types

A **variable** is a named storage location in memory. Java is **statically typed** — every variable must have a declared type.

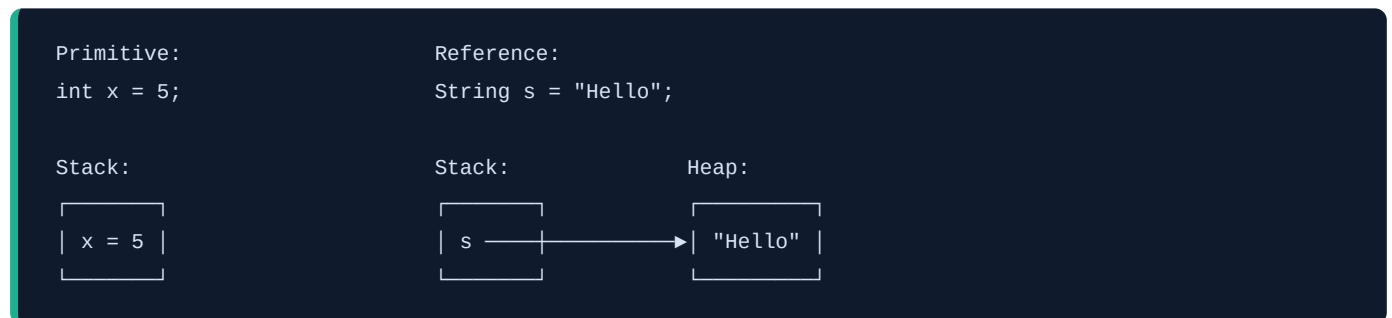
Primitive Data Types ★★★★★

Type	Size	Range (approx.)	Default Value
<code>byte</code>	1 byte	-128 to 127	0
<code>short</code>	2 bytes	-32,768 to 32,767	0
<code>int</code>	4 bytes	~-2.1B to 2.1B	0
<code>long</code>	8 bytes	~-9.2×10 ¹⁸ to 9.2×10 ¹⁸	0L
<code>float</code>	4 bytes	~7 decimal digits precision	0.0f
<code>double</code>	8 bytes	~15 decimal digits precision	0.0d
<code>char</code>	2 bytes	single Unicode character	'\u0000'
<code>boolean</code>	1 bit (JVM-dependent)	<code>true</code> / <code>false</code>	false

Key fact: Primitives store the **actual value** directly, unlike objects which store a **reference** to heap memory.

Reference Data Types

Anything that's **not** a primitive: classes, interfaces, arrays, `String` — these store a **reference (address)** to an object on the heap, not the value itself.



Identifiers, Keywords, Literals, Constants

- **Identifiers** — names for variables/methods/classes (must start with a letter, `_`, or `$`; case-sensitive)
- **Keywords** — reserved words with special meaning (`class`, `public`, `static`, `if` ...) — cannot be used as identifiers
- **Literals** — fixed values written directly in code: `42`, `3.14`, `'A'`, `"text"`, `true`
- **Constants** — declared with `final`, cannot be reassigned: `final double PI = 3.14159;`

Type Casting

```
// Widening (implicit) - smaller type to bigger type, safe, automatic
int a = 10;
double b = a;           // int → double, no data loss

// Narrowing (explicit) - bigger type to smaller type, may lose data
double c = 9.7;
int d = (int) c;        // d = 9 (decimal part truncated, not rounded!)
```

Operators (quick reference)

Category	Operators
Arithmetic	<code>+ - * / %</code>
Relational	<code>== != > < >= <=</code>
Logical	<code>&& \ \ !</code>
Assignment	<code>= += -= *= /=</code>
Unary	<code>++ -- + - !</code>
Bitwise	<code>& \ ^ ~ << >> >>></code>
Ternary	<code>condition ? valueIfTrue : valueIfFalse</code>

Wrapper Classes, Autoboxing, Unboxing ★★★★★

Every primitive type has a corresponding **wrapper class** (object version): `int` → `Integer`, `double` → `Double`, `boolean` → `Boolean`, etc.

```
int primitive = 10;
Integer wrapped = primitive;    // Autoboxing: primitive → object (automatic)
int backToPrimitive = wrapped;  // Unboxing: object → primitive (automatic)
```

Why wrapper classes exist: Collections (`ArrayList`, `HashMap`, etc.) can only store **objects**, not primitives. Wrapper classes let primitives be used in generic collections.

```
List<Integer> nums = new ArrayList<>();
nums.add(5);    // int 5 is autoboxed into Integer
```

Interview trap ★★★★★: `Integer` caches values from **-128 to 127**. So `Integer a = 100; Integer b = 100;` gives `a == b` → `true` (same cached object), but `Integer a = 200; Integer b = 200;` gives `a == b` → `false` (different objects outside the cache range)! Always use `.equals()` for wrapper comparison.

`var` keyword (Java 10+) ★★★

```
var name = "Riya";    // compiler infers String
var count = 10;       // compiler infers int
```

`var` is **not** a dynamic type — Java is still statically typed. The compiler infers the type at **compile time** based on the assigned value, and it can never change afterward. `var` cannot be used for fields, method parameters, or return types — only local variables with an initializer.

Command Line Arguments

```
public class Main {
    public static void main(String[] args) {
        System.out.println(args[0]);    // first command-line argument
    }
}
// Run with: java Main hello world → args = ["hello", "world"]
```

Part 6 — Control Flow



Decision Making

```
if (score >= 90) {
    grade = "A";
} else if (score >= 75) {
    grade = "B";
} else {
    grade = "C";
}
```

Traditional switch vs Enhanced switch (Java 14+) ★★★★★

```
// Traditional switch - needs 'break' to avoid fall-through
switch (day) {
    case 1: dayName = "Monday"; break;
    case 2: dayName = "Tuesday"; break;
    default: dayName = "Unknown";
}

// Enhanced switch expression - no fall-through, returns a value
String dayName = switch (day) {
    case 1 -> "Monday";
    case 2 -> "Tuesday";
    default -> "Unknown";
};

// yield - used when a switch branch needs multiple statements
int result = switch (grade) {
    case "A" -> 4;
    case "B" -> {
        System.out.println("Good job");
        yield 3;           // yield returns the value from the block
    }
    default -> 0;
};
```

Pattern Matching for switch (Java 21) ★★★★★

```
Object obj = 42;
String desc = switch (obj) {
    case Integer i when i > 0 -> "Positive integer: " + i;
    case Integer i             -> "Non-positive integer";
    case String s              -> "String: " + s;
    default                    -> "Unknown type";
};
```

Loops

for loop	Use when: iteration count is known
----------	------------------------------------

while loop	Use when: condition checked BEFORE each iteration
------------	---

do-while loop	Use when: body must run AT LEAST once
---------------	---------------------------------------

```
for (int i = 0; i < 5; i++) { ... }           // classic for
for (int num : numbers) { ... }              // enhanced for (for-each)
while (condition) { ... }                   // pre-tested loop
do { ... } while (condition);                // post-tested loop (runs ≥ 1 time)
```

break, continue, return, Labels

Keyword	Effect
<code>break</code>	Exits the current loop/switch entirely
<code>continue</code>	Skips the rest of the current iteration, moves to the next one
<code>return</code>	Exits the current method, optionally returning a value
Labels	Named loops (<code>outer:</code>) — allow <code>break</code> / <code>continue</code> to target a specific outer loop from within nested loops

```
outer:
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        if (j == 1) continue outer;    // skips to next 'i', not just next 'j'
        System.out.println(i + ", " + j);
    }
}
```

Interview Questions — Part 5-6

Q1. What is the difference between `==` and `.equals()` for wrapper classes like `Integer`? ★★★★★ Frequently Asked

- **Ideal Answer:** `==` compares references for objects, including wrapper classes. Java caches `Integer` objects in the range -128 to 127 (Integer Cache), so `==` may accidentally return `true` for small values but `false` for larger ones — this inconsistency is why `.equals()` should always be used for value comparison.
- **Why asked:** A classic "gotcha" question that catches unprepared candidates off guard.
- **Common mistake:** Assuming `==` always works for value comparison because it "worked" in a small test case.
- **Difficulty:** Intermediate

Q2. What is the difference between traditional switch and switch expressions? ★★★★★

- **Ideal Answer:** Traditional switch is a statement — it doesn't return a value and needs explicit `break` to prevent fall-through between cases. Switch expressions (Java 14+) can directly return a value, use `->` syntax with no fall-through, and use `yield` for multi-statement branches.
- **Difficulty:** Intermediate

Q3. Why does `var` not make Java dynamically typed? ★★★★★

- **Ideal Answer:** `var` only tells the compiler to infer the variable's type from its initializer at compile time. Once inferred, the type is fixed and cannot change — this is different from dynamically typed languages where a variable's type can change at runtime.
- **Difficulty:** Basic

Part 7 — Methods



Anatomy of a Method

```
public    static    int    add(int a, int b) {    return a + b;    }
|         |         |     |         |         |
access   modifier  return name  parameters    method body
level                                     type
```

- **Parameters** — variables listed in the method signature (`a` , `b`)
- **Arguments** — actual values passed when calling the method (`add(3, 5)` → `3` and `5` are arguments)

Pass-by-Value in Java ★★★★★

Java is always pass-by-value — never pass-by-reference. This is one of the most misunderstood (and most-asked) Java concepts.

- For **primitives**: the actual value is copied. Changes inside the method don't affect the original.
- For **objects**: the **reference (address)** is copied — not the object itself. So the method can modify the object's *internal state* through that reference, but it **cannot** make the original variable point to a different object.

```
void modify(int x) { x = 100; }           // copy of primitive - no effect outside
void modifyObj(int[] arr) { arr[0] = 100; } // modifies the SAME array via copied reference
void reassign(int[] arr) { arr = new int[]{9,9}; } // reassigns LOCAL copy only - no effect outside

int num = 5;
modify(num);
System.out.println(num);                // 5 - unchanged

int[] array = {1, 2, 3};
modifyObj(array);
System.out.println(array[0]);            // 100 - the object's content WAS changed

reassign(array);
System.out.println(array[0]);            // still 100 - reassignment didn't affect caller's reference
```

```

Caller's array reference:  arr —————> [1,2,3]   (Heap)
Method's copied reference: arr —————>      ▲
                           | same object
                           |
                           | modifying arr[0] changes THIS shared object

```

But: `arr = new int[]{9,9}` just makes the LOCAL copy point elsewhere,
the caller's original reference is untouched.

Method Overloading (recap) ★★★★★

Same method name, different parameter list, resolved at **compile time**.

```
int add(int a, int b) { return a + b; }
double add(double a, double b) { return a + b; }
```

Varargs

Allows a method to accept a **variable number of arguments** of the same type, treated internally as an array.

```
int sum(int... numbers) {           // numbers is treated as int[]
    int total = 0;
    for (int n : numbers) total += n;
    return total;
}

sum();           // 0 arguments - OK
sum(1, 2, 3);   // 3 arguments - OK
```

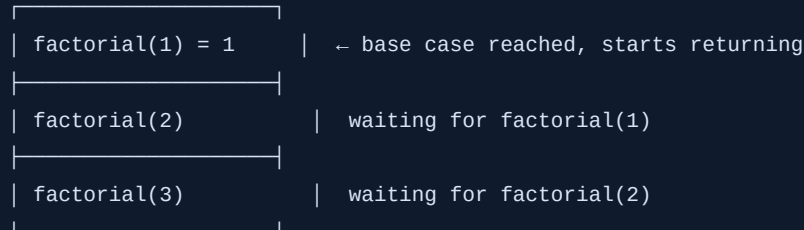
Rule: varargs must be the **last** parameter in the method signature.

Recursion and Stack Frames ★★★★★

A method that calls **itself**. Each call creates a new **stack frame** (its own local variables and return address) on the call stack.


```
int factorial(int n) {
    if (n <= 1) return 1;           // base case - stops recursion
    return n * factorial(n - 1);    // recursive case
}
```

Call Stack for factorial(3):



Stack grows downward as calls nest, unwinds as they return

Danger: Missing/incorrect base case → infinite recursion → `StackOverflowError`.

Interview Questions — Part 7

Q1. Is Java pass-by-value or pass-by-reference? ★★★★★ Frequently Asked

- **Ideal Answer:** Java is always pass-by-value. For objects, the *value of the reference* (i.e., the memory address) is copied — so the method can modify the object's internal state, but reassigning the parameter inside the method never affects the caller's original reference.
- **Why asked:** Extremely common and frequently misunderstood — separates candidates with real understanding from those who memorized "Java is pass by reference for objects" incorrectly.
- **Common mistake:** Saying Java is "pass-by-reference for objects" — technically incorrect; it's pass-by-value of the reference.
- **Difficulty:** Intermediate

Q2. What happens if a recursive method has no base case? ★★★★★

- **Ideal Answer:** The method keeps calling itself indefinitely, continuously pushing new stack frames onto the call stack until memory is exhausted, causing a `StackOverflowError`.
- **Difficulty:** Basic

Q3. Can varargs be combined with other parameters? ★★★★★

- **Ideal Answer:** Yes, but the varargs parameter must be the last one in the parameter list, since the compiler needs to know where fixed parameters end and the variable-length ones begin.
- **Difficulty:** Basic

Part 8 — Object-Oriented Programming



What is OOP and Why It Was Invented

Procedural programming (like C) organizes code as functions operating on separate, often global, data. As programs grew large, this caused **code duplication, poor data security, and maintenance nightmares** — any function could corrupt shared data, and there was no natural way to model real-world entities.

OOP solves this by bundling **data (attributes)** and **behavior (methods)** together into **objects**, modeled after real-world things.

Procedural Problem	OOP Solution
No data protection	Encapsulation — private fields, controlled access
Code duplication	Inheritance — reuse via parent-child classes
Poor real-world mapping	Objects naturally model real entities (Customer, Order)
Tight coupling	Abstraction/Interfaces — depend on contracts, not implementations

Real-world analogy: A **car** has attributes (color, speed) and behavior (start, brake) bundled into one unit — you don't manage the engine's internals separately from the color as unrelated global data.

Classes and Objects

Term	Meaning
Class	Blueprint/template defining structure and behavior
Object	An actual instance of a class, created with <code>new</code> , living on the heap
Reference Variable	A stack variable holding the <i>address</i> of an object, not the object itself
State	The current values of an object's attributes
Behavior	What an object can do (its methods)
Identity	An object's unique existence in memory, distinct even from identical-state objects

```
class Student {
    String name;           // attribute (state)
    void study() { ... }   // method (behavior)
}

Student s1 = new Student(); // 's1' (stack) references a new Student object (heap)
```



Object Lifecycle & Garbage Collection ★★★★★

1. Object created via `new` → allocated on the heap
2. Used via reference variable(s)
3. When **no reference points to it anymore**, it becomes eligible for **Garbage Collection**
4. JVM's GC automatically reclaims that memory at some later point (not immediately/deterministically)

Constructors ★★★★★

A constructor initializes an object at creation time. Same name as the class, no return type.

```
class Student {
    String name;
    Student() { name = "Unknown"; }           // default constructor
    Student(String name) { this.name = name; } // parameterized constructor

    Student() { this("Unknown"); }           // constructor chaining via this()
}
```

```
class Person {
    Person(String type) { System.out.println("Person: " + type); }
}
class Employee extends Person {
    Employee() {
        super("Staff"); // must be FIRST statement - calls parent constructor
        System.out.println("Employee created");
    }
}
```

Copy Constructor concept (Java has no built-in copy constructor like C++, but the pattern is common):

```
class Point {
    int x, y;
    Point(Point other) { // manually written "copy constructor"
        this.x = other.x;
        this.y = other.y;
    }
}
```

Best practices: Validate inputs inside constructors; avoid heavy logic in constructors; use constructor chaining (`this()`) to avoid duplicated initialization code.

The Four Pillars of OOP ★★★★★

Encapsulation	Abstraction	Inheritance	Polymorphism
"Hide data"	"Hide complexity"	"Reuse via parent-child"	"Many forms, one interface"

Encapsulation

Bundling data + methods, restricting direct access via **access modifiers**, so state can only change through validated, controlled methods.

```
class BankAccount {
    private double balance;                // hidden from outside
    public double getBalance() { return balance; }    // controlled read
    public void withdraw(double amt) {
        if (amt > 0 && amt <= balance) balance -= amt; // business rule enforced
    }
}
```

Modifier	Same Class	Same Package	Subclass (diff pkg)	Everywhere
private	✓	✗	✗	✗
default	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

Abstraction

Hiding implementation **complexity**, showing only essential features — achieved via **abstract classes** (partial implementation allowed) and **interfaces** (pure contract, traditionally).

```
abstract class Shape {
    abstract double area();                // must be implemented by subclasses
    void printArea() { System.out.println(area()); } // shared concrete method
}
interface Payable {
    void pay(double amount);                // implicit public abstract
}
```

Abstraction vs Encapsulation: Abstraction hides complexity (design-level, "what to show"). Encapsulation hides data (implementation-level, "how to protect it").

Inheritance

Child class reuses parent's fields/methods — models an **IS-A** relationship.

```
class Animal { void eat() { ... } }
class Dog extends Animal { void bark() { ... } } // Dog IS-A Animal
```

Type	Structure	Java Support
Single	$A \rightarrow B$	✓
Multilevel	$A \rightarrow B \rightarrow C$	✓
Hierarchical	$A \rightarrow B, A \rightarrow C$	✓
Multiple (classes)	$A, B \rightarrow C$	✗ Diamond Problem
Multiple (interfaces)	$A, B \rightarrow \text{implemented by } C$	✓

Why no multiple class inheritance: If class `D` inherited from both `B` and `C`, and both override a method from a common ancestor `A` differently, the compiler can't resolve which version `D` should use (the **Diamond Problem**). **Interfaces solve this** because they don't carry conflicting state — and Java 8+ forces explicit resolution if two interfaces have conflicting **default** methods.

Polymorphism

Compile-time (Overloading): same name, different parameters, resolved by the compiler. **Runtime (Overriding):** same signature, redefined in subclass, resolved via **Dynamic Method Dispatch** at runtime.

```
class Animal { void sound() { System.out.println("Generic sound"); } }
class Cat extends Animal { void sound() { System.out.println("Meow"); } }

Animal a = new Cat();
a.sound();    // "Meow" - JVM checks the ACTUAL object type at runtime, not reference type
```

This runtime resolution is called **Dynamic Method Dispatch** or **Late Binding** — the JVM decides which overridden method to invoke based on the real object, not the declared reference type.

Interview Questions — Part 8

Q1. Why was OOP invented? ★★★★★ Frequently Asked

- **Ideal Answer:** Procedural programming separates data and functions, causing poor data protection, code duplication, and maintenance difficulty as programs scale. OOP bundles data and behavior into objects, models real-world entities naturally, and controls access through encapsulation.
- **Difficulty:** Basic

Q2. Explain Dynamic Method Dispatch with an example. ★★★★★

- **Ideal Answer:** When a parent-type reference points to a child object and an overridden method is called, the JVM determines which class's method to execute based on the actual object's type at runtime, not the reference's declared type. E.g., `Animal a = new Cat(); a.sound();` calls `Cat`'s `sound()`.
- **Why asked:** Core to understanding how runtime polymorphism actually works.
- **Difficulty:** Advanced

Q3. Why doesn't Java support multiple inheritance with classes? ★★★★★

- **Ideal Answer:** To avoid the Diamond Problem — ambiguity in which parent's method a subclass should inherit when two parents provide conflicting implementations. Interfaces solve this because Java 8+ forces explicit resolution of conflicting default methods.
- **Difficulty:** Intermediate

Q4. Scenario: You have `Employee emp = new Manager();` — what determines which `getSalary()` runs if both classes define it? ★★★★★

- **Ideal Answer:** `Manager`'s version runs, due to dynamic method dispatch — the JVM checks the actual object type (`Manager`), not the reference type (`Employee`).
- **Difficulty:** Intermediate

Part 9 — Java Keywords



Keyword	Meaning	Internal Working / Interview Angle
<code>this</code>	Refers to the current object	Disambiguates field vs parameter names; can call another constructor via <code>this()</code>
<code>super</code>	Refers to the parent class	Accesses parent fields/methods; calls parent constructor — must be the first statement
<code>static</code>	Belongs to the class, not an instance	Stored once in memory (metaspace), shared by all objects; accessed via <code>ClassName.member</code>
<code>final</code>	Prevents modification	<code>final</code> variable = constant; <code>final</code> method = cannot be overridden; <code>final</code> class = cannot be extended
<code>instanceof</code>	Checks an object's type at runtime	Prevents unsafe downcasting/ <code>ClassCastException</code> ; Java 17+ supports pattern matching (<code>obj instanceof String s</code>)
<code>new</code>	Allocates memory for an object on the heap	Triggers constructor call after memory allocation

```
class Counter {
    static int count = 0;        // ONE copy shared across all objects
    Counter() { count++; }
}
// Every 'new Counter()' increments the SAME static 'count'
```

```
Object obj = "hello";
if (obj instanceof String s) {    // Java 17+ pattern matching
    System.out.println(s.length()); // 's' auto-cast, ready to use
}
```

Part 10 — Advanced OOP Concepts



Nested and Inner Classes

Type	Description
Static Nested Class	Declared <code>static</code> inside another class; doesn't need an outer instance to exist
Inner Class (non-static)	Tied to an instance of the outer class; can access outer class's instance members
Anonymous Class	A one-off class with no name, defined and instantiated in a single expression — often used for quick implementations of interfaces

```

class Outer {
    static class StaticNested { }           // doesn't need Outer instance
    class Inner { }                         // needs Outer instance: outer.new Inner()

    Runnable r = new Runnable() {          // anonymous class implementing Runnable
        public void run() { System.out.println("Running"); }
    };
}

```

Interfaces — Modern Features ★★★★★

```

interface Vehicle {
    void start();                          // abstract method (implicit)

    default void honk() {                  // default method (Java 8+) - has a body
        System.out.println("Beep!");
    }

    static void info() {                   // static method (Java 8+)
        System.out.println("Vehicles must implement start()");
    }

    private void log() { System.out.println("log"); } // private interface method (Java 9+) - helper for
}

```

Concept	Meaning
Functional Interface	An interface with exactly one abstract method — can be implemented via a lambda expression. Marked with <code>@FunctionalInterface</code> (e.g., <code>Runnable</code> , <code>Comparator</code>)
Marker Interface	An interface with no methods at all — used purely to "tag" a class with metadata (e.g., <code>Serializable</code> , <code>Cloneable</code>)
Default Methods	Let interfaces provide a method body, enabling backward-compatible interface evolution without breaking existing implementers
Private Interface Methods	Helper methods (Java 9+) shared between default methods within the same interface, not exposed to implementers

```

@FunctionalInterface
interface Calculator { int operate(int a, int b); }

Calculator add = (a, b) -> a + b; // lambda implementing the functional interface

```

The Object Class — Recap ★★★★★

Every class implicitly extends `Object`. Key methods: `equals()`, `hashCode()`, `toString()`, `clone()`.

Contract: If two objects are `.equals()`, they **must** have the same `hashCode()`. Breaking this rule silently corrupts behavior in `HashMap` / `HashSet` (duplicate entries, failed lookups).

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Point p)) return false;
    return x == p.x && y == p.y;
}

@Override
public int hashCode() { return Objects.hash(x, y); }
```

Covariant Return Types

An overriding method may return a **narrower (subtype)** return type than the parent's declared return type.

```
class Animal { Animal reproduce() { return new Animal(); } }
class Dog extends Animal {
    @Override Dog reproduce() { return new Dog(); }    // narrower return type allowed
}
```

Sealed Classes (Java 17) ★★★★★

Restrict **which classes** are allowed to extend/implement a type — giving controlled, exhaustive inheritance.

```
sealed interface Shape permits Circle, Square, Triangle {}
final class Circle implements Shape { }
final class Square implements Shape { }
final class Triangle implements Shape { }
```

This lets the compiler know the **complete** set of subtypes, enabling exhaustive `switch` pattern matching without a `default` branch.

Records (Java 17) — Recap

```
record Point(int x, int y) { }
// Auto-generates: constructor, x(), y() accessors, equals(), hashCode(), toString()
```

Records are **implicitly final** and **immutable** — designed for simple data-carrier classes.

SOLID Principles — Java Perspective ★★★★★

Principle	Idea	Java Example
Single Responsibility	A class should have one reason to change	Separate <code>ReportGenerator</code> from <code>ReportPrinter</code> instead of one giant class
Open/Closed	Open for extension, closed for modification	Use interfaces/abstract classes so new behavior is added via new subclasses, not by editing existing tested code
Liskov Substitution	Subclass objects must be usable wherever the parent is expected	A <code>Square</code> extends <code>Rectangle</code> that breaks <code>setWidth()</code> / <code>setHeight()</code> behavior violates this
Interface Segregation	Prefer small, specific interfaces over one large one	Split a bulky <code>Worker</code> interface into <code>Workable</code> , <code>Eatable</code> rather than forcing unused methods
Dependency Inversion	Depend on abstractions, not concrete classes	A <code>Service</code> depends on a <code>Repository</code> interface, and Spring injects the actual implementation

Interview Questions — Part 9-10

Q1. What is a functional interface? Give an example. ★★★★★

- **Ideal Answer:** An interface with exactly one abstract method, which allows it to be implemented using a lambda expression. Examples: `Runnable` (`run()`), `Comparator<T>` (`compare()`), `Callable<T>` (`call()`).
- **Why asked:** Foundational for understanding Java 8+ functional-style code and the Streams API.
- **Difficulty:** Intermediate

Q2. What is a marker interface? Give an example. ★★★

- **Ideal Answer:** An interface with no methods, used to "tag" a class with metadata that the JVM or a framework checks via `instanceof` . Example: `Serializable` marks a class as eligible for serialization.
- **Difficulty:** Basic

Q3. Why were default methods added to interfaces in Java 8? ★★★★★

- **Ideal Answer:** To allow interface evolution — adding new methods to an interface without breaking every existing class that implements it. This was essential for adding new Streams-related methods to existing collection interfaces without breaking the entire ecosystem.
- **Difficulty:** Intermediate

Q4. What is a sealed class and why use one? ★★★★★

- **Ideal Answer:** A sealed class/interface (Java 17+) restricts which classes may extend/implement it via a `permits` clause. This gives the compiler complete knowledge of all subtypes, enabling safer, exhaustive pattern matching and more predictable domain modeling.
- **Difficulty:** Advanced

Part 11 — Frequently Confused Concepts



JDK vs JRE vs JVM

JDK	JRE	JVM
Development kit (compiler + tools)	Runtime environment (libraries + JVM)	The execution engine itself
For developers	For running Java apps	Core component inside JRE
Superset of JRE	Superset of JVM	Platform-specific

Class vs Object

Class	Object
Blueprint, no memory until instantiated	Instance, occupies heap memory
Defines structure/behavior	Has actual state (values)

Object vs Reference

Object	Reference
The actual data, lives on the heap	A variable (on the stack) holding the object's address
Created with <code>new</code>	Declared with a type: <code>Student s</code>

Abstract Class vs Interface

Abstract Class	Interface
Can have both abstract & concrete methods, instance fields	Mostly constants + method signatures (default/static allowed since Java 8)
Single inheritance only	A class can implement multiple interfaces
Can have a constructor	Cannot have a constructor
Use for shared base state + "IS-A"	Use for a pure capability/contract

Method Overloading vs Overriding

Overloading	Overriding
Same class, different parameters	Parent-child classes, same signature
Compile-time resolution	Runtime resolution (dynamic dispatch)
Return type can differ freely	Return type must be same/covariant

this vs super

this	super
Refers to current object	Refers to the parent class
Calls sibling constructor via <code>this()</code>	Calls parent constructor via <code>super()</code>

== vs .equals()

==	.equals()
Compares references (memory address)	Compares logical/content equality (if overridden)
For primitives, compares actual value	N/A for primitives

Primitive vs Reference Types

Primitive	Reference
Stores actual value	Stores an address to heap memory
Fixed set: <code>int</code> , <code>char</code> , <code>boolean</code> ...	Classes, interfaces, arrays, <code>String</code>
Cannot be <code>null</code>	Can be <code>null</code>

Composition vs Inheritance

Composition (HAS-A)	Inheritance (IS-A)
Loosely coupled, flexible at runtime	Tightly coupled to parent's implementation
"Favor composition over inheritance" — safer for large systems	Use only for genuine type hierarchies

Association vs Aggregation vs Composition

	Association	Aggregation	Composition
Strength	Weakest	Weak HAS-A	Strong HAS-A
Lifecycle	Independent	Independent	Dependent
Example	Doctor-Patient	Department-Professor	House-Room

Static vs Instance Members

Static	Instance
One copy shared by the class	Separate copy per object
Accessed via <code>ClassName.member</code>	Accessed via <code>object.member</code>
Loaded once at class-loading time	Created per <code>new</code> call

Record vs Class

Record	Regular Class
Implicitly final, immutable	Mutable by default
Auto-generates constructor/getters>equals/hashCode/toString	Must write these manually
Best for simple data carriers	Best for full behavior-rich objects

Sealed Class vs Final Class

Sealed Class	Final Class
Allows a specific, controlled set of subclasses (<code>permits</code>)	Allows no subclasses at all
Enables exhaustive pattern matching	Simply prevents extension

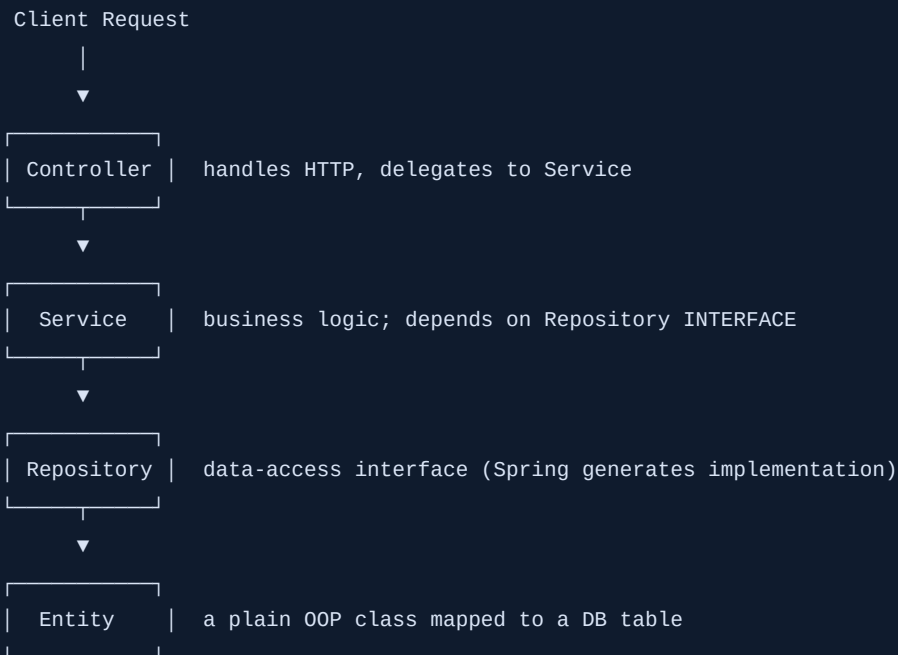
Part 12 — Java in Real Projects

★★★★★

Domain Mapping

Domain	How OOP Concepts Map
Banking System	<code>Account</code> (class) → <code>SavingsAccount</code> / <code>CurrentAccount</code> (inheritance); <code>Transaction</code> (composition inside Account); interest calculation via an interface (polymorphism)
Hospital Management	<code>Person</code> → <code>Doctor</code> , <code>Patient</code> (inheritance); <code>Appointment</code> (association); <code>MedicalRecord</code> (composition)
E-commerce	<code>Product</code> , <code>Cart</code> (composition); <code>PaymentMethod</code> interface implemented by <code>CreditCard</code> , <code>UPI</code> , <code>Wallet</code>
Authentication System	<code>User</code> entity; <code>Role</code> -based access via interfaces/enums; encapsulated password hashing logic
REST APIs	Each resource modeled as a class (<code>UserController</code> , <code>UserService</code>); DTOs decouple internal entities from API responses

OOP – Spring Boot Mapping ★★★★★



Spring Concept	OOP Concept
<code>@Entity</code> class	Plain class with attributes, mapped to a DB table
<code>DTO</code>	A plain class used to transfer data between layers (abstraction — hides internal entity structure)
Interface + Implementation (<code>UserService</code> / <code>UserServiceImpl</code>)	Abstraction + polymorphism
<code>@Autowired</code> / constructor injection	Composition — a class "has-a" injected dependency instead of creating it with <code>new</code>
Dependency Injection / IOC	Dependency Inversion Principle in action — Spring creates and wires objects instead of your code doing it
Bean	Just an object instance managed by the Spring container

Microservices: Each service is typically its own Spring Boot app, internally structured with the same Controller → Service → Repository → Entity OOP layering, communicating with other services via REST/messaging.

Part 13 — Placement Focus Summary



Topic	Importance	Why Companies Ask
JDK vs JRE vs JVM	★★★★★	Universal opening question — tests basic architecture understanding
Java 8 features (Lambdas, Streams)	★★★★★	Nearly every modern Java codebase uses these
Pass-by-value	★★★★★	Frequently misunderstood — tests real depth
Four Pillars of OOP	★★★★★	Foundation for all design discussions
equals()/hashCode() contract	★★★★★	Common real bug source in production code
Functional Interfaces & Lambdas	★★★★★	Needed to understand modern Java & Streams
SOLID Principles	★★★★★	Tests design maturity for real backend systems
OOP in Spring Boot	★★★★★	Bridges academic knowledge to real job responsibilities
Records & Sealed Classes	★★★	Signals awareness of modern Java (17/21)

Part 14 — Common Java Mistakes



Mistake	Why It Happens	How to Avoid It
Using <code>==</code> instead of <code>.equals()</code>	Confusing reference comparison with value comparison, especially with wrapper classes/Strings	Always use <code>.equals()</code> for objects; understand the Integer/String cache trap
Public fields	Convenience during quick prototyping	Keep fields <code>private</code> ; expose via validated getters/setters
Improper/excessive inheritance	Treating inheritance as "the" way to reuse code	Prefer composition when there's no true IS-A relationship
Constructor mistakes	Forgetting to call <code>super()</code> correctly, or leaving fields uninitialized	Always validate inputs; use constructor chaining to avoid duplication
<code>NullPointerException</code>	Calling a method on an object that was never initialized or came back <code>null</code> from a lookup	Check for <code>null</code> , use <code>Optional</code> where appropriate, initialize defensively
Incorrect overriding	Wrong method signature accidentally creates an overload instead of an override	Always use <code>@Override</code> — the compiler will flag mismatches
Poor naming conventions	Rushed code, unclear intent	Use descriptive <code>camelCase</code> for variables/methods, <code>PascalCase</code> for classes
Not following encapsulation	Exposing internal collections/arrays directly	Return defensive copies, or unmodifiable views, of internal mutable state

```
// Common mistake: leaking internal mutable state
class Team {
    private List<String> members = new ArrayList<>();
    public List<String> getMembers() { return members; }    // BAD - caller can mutate internal list!
    public List<String> getMembersSafe() {
        return Collections.unmodifiableList(members);      // GOOD - protects internal state
    }
}
```

Part 15 — Final Revision



One-Page Quick Revision

Term	One-line Definition
JVM	Executes bytecode
JRE	JVM + standard libraries (to run Java apps)
JDK	JRE + development tools (to write/compile Java apps)
Bytecode	Platform-neutral compiled form of Java source code
JIT Compiler	Converts "hot" bytecode to native machine code at runtime for speed
Class Loader	Loads <code>.class</code> files into JVM memory on demand
Autoboxing/Unboxing	Automatic conversion between primitives and their wrapper classes
<code>var</code>	Compile-time type inference for local variables (still statically typed)
Pass-by-value	Java always copies the value — for objects, it copies the reference itself
Encapsulation	Bundling data + methods, restricting access via modifiers
Abstraction	Hiding implementation complexity, exposing essentials
Inheritance	Child reuses parent's fields/methods (IS-A)
Polymorphism	One interface, many implementations (overloading + overriding)
Dynamic Method Dispatch	Runtime resolution of overridden methods based on actual object type
Functional Interface	Interface with exactly one abstract method (lambda target)
Record	Concise, immutable data-carrier class (Java 17+)
Sealed Class	Restricts which classes may extend/implement it

Java Foundations Cheat Sheet

- **Primitives:** `byte`, `short`, `int`, `long`, `float`, `double`, `char`, `boolean` — store actual values

- **Reference types:** classes, interfaces, arrays, `String` — store addresses to heap objects
- **Widening cast:** automatic, safe (`int` → `double`)
- **Narrowing cast:** explicit, may lose data (`double` → `int` truncates)
- **Wrapper cache:** `Integer` caches -128 to 127 — `==` is unreliable outside this range
- `var` : type inferred at compile time, fixed forever after, local variables only

OOP Cheat Sheet

- **Pillars mnemonic:** *E-A-I-P* → "Every Aspiring Interviewee Practices"
- **Access modifiers (widest to narrowest):** `public` > `protected` > `default` > `private`
- **Overloading:** compile-time, same class, different params
- **Overriding:** runtime, parent-child, same signature
- **equals()/hashCode() rule:** equal objects MUST have equal hashCodes (not vice versa)
- **Composition > Inheritance** for flexible, loosely-coupled design
- **Diamond Problem:** why Java has no multiple class inheritance; interfaces solve it

Memory Tricks

- **JDK ⊃ JRE ⊃ JVM** — like Russian nesting dolls, each contains the previous
- **SOLID:** *S-O-L-I-D* → Single, Open/Closed, Liskov, Interface Segregation, Dependency Inversion
- **Pass-by-value for objects:** "the address is copied, not the object" — you can change what's *inside* the house, but you can't make the caller's mailbox point to a different house

Topic Dependency Map

```
graph TD
    A[Java Architecture (JDK/JRE/JVM, Bytecode)] --> B[Java Fundamentals (variables, types, operators)]
    B --> C[Control Flow (if/switch/loops) → Methods (params, pass-by-value, recursion)]
    C --> D[OOP Basics (Class, Object, Constructors)]
    D --> E[Four Pillars → Encapsulation → Abstraction → Inheritance → Polymorphism]
    E --> F[Java Keywords (this, super, static, final)]
    F --> G[Advanced OOP (Interfaces, Records, Sealed Classes, Object class methods)]
    G --> H[SOLID Principles → Real Project Usage → Spring Boot Mapping]
    H --> I[Interview Practice (Top 50 Questions)]
```

30-Minute Last-Minute Revision Checklist

- ☐ Can you explain JDK vs JRE vs JVM in one breath?
- ☐ Can you explain bytecode and WORA (Write Once, Run Anywhere)?
- ☐ Can you name Java 8's biggest features and why they matter?
- ☐ Can you explain pass-by-value for objects with a code example?
- ☐ Can you define all 4 pillars of OOP with one example each?
- ☐ Can you explain the equals()/hashCode() contract?
- ☐ Can you explain why Java has no multiple class inheritance?
- ☐ Can you explain the difference between abstract class and interface?
- ☐ Can you explain a functional interface and give an example?
- ☐ Can you connect OOP concepts to Spring Boot's Controller/Service/Repository layers?

Top 50 Java Foundations & OOP Interview Questions

- 1. What is Java?** — A platform-independent, object-oriented programming language that compiles to bytecode executed by the JVM.
- 2. JDK vs JRE vs JVM?** — JVM executes bytecode; JRE = JVM + libraries; JDK = JRE + development tools.
- 3. What is bytecode?** — Platform-neutral intermediate code produced by `javac`, executed by the JVM.
- 4. Explain WORA.** — Bytecode is portable across platforms; only the JVM implementation is platform-specific.

5. **What is the JIT compiler?** — Compiles frequently executed ("hot") bytecode into native machine code at runtime for speed.
6. **What is the Class Loader?** — Loads `.class` files into JVM memory on demand, following a parent-delegation hierarchy.
7. **Is Java 100% object-oriented?** — No, it has primitive types that aren't objects.
8. **What's new in Java 8?** — Lambdas, Streams API, `Optional`, default interface methods, new Date/Time API.
9. **What's new in Java 17?** — Sealed classes, Records, pattern matching for `instanceof`, text blocks.
10. **What's new in Java 21?** — Virtual threads, pattern matching for `switch`, record patterns.
11. **Difference between primitive and reference types?** — Primitives store actual values; references store heap addresses.
12. **What is autoboxing?** — Automatic conversion of a primitive into its wrapper class (e.g., `int` → `Integer`).
13. **Why does `Integer a=127; Integer b=127; a==b` give true but `200` gives false?** — Integer caching applies only to -128 to 127.
14. **What is `var` and is Java dynamically typed now?** — Local variable type inference; Java remains statically typed since the type is fixed at compile time.
15. **Difference between `==` and `.equals()`?** — `==` compares references (or primitive values); `.equals()` compares logical content.
16. **Widening vs narrowing casting?** — Widening is implicit/safe (small→big type); narrowing is explicit and may lose data.
17. **What are varargs?** — A way to pass a variable number of arguments, treated as an array internally.
18. **Is Java pass-by-value or pass-by-reference?** — Always pass-by-value; for objects, the reference's value (address) is copied.
19. **What causes a `StackOverflowError`?** — Typically infinite/unterminated recursion exhausting the call stack.
20. **What is method overloading?** — Same method name, different parameter lists, resolved at compile time.
21. **What is method overriding?** — Same signature redefined in a subclass, resolved at runtime.
22. **What is OOP and why was it invented?** — A paradigm bundling data+behavior into objects, solving procedural programming's scalability and data-protection issues.
23. **Class vs Object?** — Class is a blueprint; object is an instance with actual memory and state.
24. **What is encapsulation?** — Bundling data+methods, restricting direct access via access modifiers.
25. **What is abstraction?** — Hiding implementation complexity, exposing only essential features.
26. **Abstraction vs Encapsulation?** — Abstraction hides complexity (design); encapsulation hides data (implementation).
27. **What is inheritance?** — Child class reuses a parent's fields/methods; models an IS-A relationship.
28. **Why no multiple inheritance with classes in Java?** — Avoids the Diamond Problem's method-resolution ambiguity.
29. **How do interfaces solve multiple inheritance?** — They provide contracts without conflicting state, with explicit resolution required for default method conflicts.
30. **What is polymorphism?** — One interface/name, multiple behaviors — via overloading (compile-time) and overriding (runtime).
31. **What is Dynamic Method Dispatch?** — Runtime resolution of an overridden method based on the actual object type.
32. **Abstract class vs Interface?** — Abstract class allows partial implementation + state; interface is a pure contract (with default/static methods since Java 8).
33. **What is constructor chaining?** — Calling one constructor from another via `this()` (same class) or `super()` (parent class).
34. **Can a constructor be private? Why?** — Yes — commonly used in the Singleton pattern to control instantiation.
35. **What is the Object class?** — The root class of all Java classes; provides `equals()`, `hashCode()`, `toString()`, `clone()`.
36. **Why override `hashCode()` with `equals()`?** — To satisfy the equals-hashCode contract required for correct behavior in hash-based collections.
37. **Composition vs Inheritance?** — Composition (HAS-A, flexible) is often preferred over inheritance (IS-A, tightly coupled).
38. **Association vs Aggregation vs Composition?** — Increasing ownership strength: none → weak → strong (lifecycle-bound).
39. **What is a functional interface?** — An interface with exactly one abstract method, usable as a lambda target.
40. **What is a marker interface?** — An interface with no methods, used to tag classes with metadata (e.g., `Serializable`).
41. **Why were default methods added in Java 8?** — To let interfaces evolve without breaking existing implementing classes.
42. **What is a Record?** — A concise, immutable data-carrier class (Java 17+) that auto-generates boilerplate.
43. **What is a Sealed Class?** — Restricts which classes can extend/implement it, enabling exhaustive pattern matching.

- 44. What is covariant return type?** — An overriding method may return a subtype of the parent method's declared return type.
- 45. What is SOLID?** — Five design principles: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion.
- 46. How does OOP relate to Spring Boot?** — Controllers/Services/Repositories are OOP classes; interfaces + Dependency Injection implement the Dependency Inversion Principle.
- 47. What is Dependency Injection?** — A design pattern where an object's dependencies are provided externally (e.g., by Spring) rather than created internally with `new`.
- 48. Static vs instance members?** — Static belongs to the class (one shared copy); instance belongs to each object (separate copy per object).
- 49. What is a common NullPointerException cause and fix?** — Calling a method on an uninitialized/ `null` reference; fix with null checks or `Optional`.
- 50. Why is "favor composition over inheritance" good advice?** — Inheritance tightly couples a subclass to its parent's implementation (fragile base class problem); composition allows flexible, swappable behavior at runtime.

Key Takeaways

- Java's core strength is **portability + maturity** — bytecode + JVM make "write once, run anywhere" real, backed by 25+ years of enterprise trust.
- **Java 8 is a hard prerequisite** for modern Java interviews — Lambdas and Streams show up everywhere.
- **Pass-by-value** is the single most misunderstood mechanic — always be ready to explain it with an object example.
- The **Four Pillars work together**, not in isolation — and interviewers reward explaining *why*, not just definitions.
- Always be ready to **connect OOP theory to Spring Boot** — that's the bridge from "academic Java" to "job-ready Java."

End of Handbook 1. Continue to Handbook 2 (Collections, Exception Handling & Multithreading) to keep building your placement-ready Java foundation.